FINNEWS AI

Financial News Simplifier

A Full-Stack Generative AI Web Application

Name:- Kanisha Jamir Tamboli

Div:-A(AIML)

Roll No:-32139

Class:-SYBTECH

CONTENTS

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives of the Project

5. Scope of the Project

6. Existing System

7. Proposed System

8. System Requirements
   - Hardware Requirements
   - Software Requirements

9. System Architecture

10. Technology Stack

11. Project Features and Modules
- Financial News Retrieval
- AI-Powered News Simplification
- News Search Functionality
- Category-Based News Browsing

- Financial Term Explanation
- Responsive User Interface
- Loading Indicators and Error Handling
- API Key Security

12. System Workflow

13. System Implementation
- Frontend Implementation
- Backend Implementation
- NewsAPI Integration
- Groq AI Integration
- API Endpoints

14. Security and Error Handling

15. Testing and Results

16. Advantages of the Project

17. Limitations of the Project

18. Applications of the Project

19. Future Scope

20. Conclusion

21. References

Abstract

FinNews AI is a full-stack web application designed to make financial news easier to understand. Financial articles often contain specialised terminology, dense explanations, and market-related language that can be difficult for beginners and general readers. The project combines news retrieval with a large language model to present important information in simpler, more accessible language.

According to the supplied project README, the application retrieves news through NewsAPI and uses Groq's Llama 3.3 70B model to analyse and simplify article content. A browser-based interface allows users to fetch news, search for financial stories, browse categories, and request explanations of financial terms. A Python FastAPI backend connects the frontend to external services through API endpoints.

The main objective is educational accessibility rather than investment advice. The application aims to identify the central event, summarise key points, explain difficult concepts, and preserve important facts from the source article. The README also describes responsive design, loading states, animations, API-key protection, and error handling as project features.

This report explains the problem, objectives, proposed system, architecture, technologies, features, workflow, security considerations, testing approach, limitations, and future enhancements. The ZIP provided for this report contains a README file only; therefore, implementation details in this document are based on the documented project description and should be checked against the actual source code before final academic submission.

1. Introduction

Financial news influences public understanding of businesses, markets, inflation, interest rates, banking, employment, and the wider economy. However, many articles assume that readers already understand financial concepts and market terminology. A beginner may struggle to distinguish the main event from background details or understand why a development matters.

FinNews AI addresses this communication gap by combining a news-data source with generative artificial intelligence. Instead of requiring every reader to interpret a long article independently, the application aims to provide a concise and beginner-friendly explanation. The user can then return to the original article for full context and verification.

The application follows a client-server model. The frontend presents controls and results. The Python backend manages requests, calls the news service, communicates with the AI service, and returns responses. This separation keeps API credentials on the server side and makes the system easier to maintain.

The project is relevant to students, new investors, general readers, and professionals who want a quick explanation of financial events. It demonstrates practical integration of web development, REST APIs, external news data, and a large language model.

2. Problem Statement

Financial news is frequently written using technical vocabulary, complex sentence structures, abbreviations, and references to economic indicators. Readers who lack a financial background may find it difficult to understand the meaning and possible context of an article.

Manually researching every unfamiliar term takes time. Reading several sources can also be difficult when a reader only needs an initial overview. A conventional news listing provides headlines and links, but it may not explain complex terminology in a way that is suitable for beginners.

The problem addressed by FinNews AI is therefore: how can current financial news be retrieved and transformed into a clear, concise, and accessible explanation while keeping the original source available for verification?

The proposed solution is a web application that retrieves news from NewsAPI and uses an AI model to simplify article content. A separate term-explanation capability is documented to help users understand individual financial concepts. The system must also manage external-service failures, loading states, and API credentials responsibly.

3. Objectives and Scope

Objectives

- Retrieve recent financial news using an external news API.
- Convert complex financial articles into simpler language using an LLM.
- Help users search for news and browse news categories.
- Explain financial terms using beginner-friendly wording.
- Provide a responsive interface for different screen sizes.
- Handle API failures and display appropriate loading or error feedback.
- Keep API keys in server-side environment variables rather than frontend code.

Scope

The documented scope covers news retrieval, AI-based simplification, term explanations, a web interface, and backend API integration. It is an informational and educational tool. It does not claim to predict market movements, guarantee the accuracy of an investment outcome, or provide personalised financial recommendations.

Features such as user accounts, saved articles, reading history, sentiment analysis, multilingual support, and database storage are listed as future improvements in the README, not as confirmed current features.

4. Existing System and Proposed System

Existing approach

A reader typically opens a news website, reads an article, searches unfamiliar words separately, and compares other articles if additional context is needed. This approach can be useful but may require considerable time and financial knowledge.

Proposed approach

FinNews AI brings several steps into one interface. It retrieves financial news, provides a simplified AI-generated explanation, and offers a way to explain financial terms. This reduces the effort required to obtain an initial understanding while preserving the importance of consulting the original source.

Expected benefits

- Less effort spent decoding difficult language.
- A faster first-pass understanding of a financial event.
- A beginner-friendly route to learning financial vocabulary.
- A single interface for news discovery and explanation.

The proposed system supports comprehension, not replacement of professional financial research. AI-generated summaries may omit nuance or misinterpret a source, so users should verify important claims in the original article.

5. System Architecture

The README describes a full-stack architecture with a browser interface, a FastAPI backend, NewsAPI for article retrieval, and Groq's Llama 3.3 70B model for simplification.

Logical flow

User → Frontend interface → FastAPI backend → NewsAPI → Article data → AI service (Groq / Llama 3.3 70B) → Simplified explanation → Frontend display → User

The frontend collects user actions and presents the response. FastAPI acts as the application layer and provides endpoints. The news service supplies article data, while the AI service transforms selected content into a simpler explanation. The final response is returned to the interface.

This architecture separates presentation, application logic, news retrieval, and AI processing. Such separation can make debugging and maintenance easier because each responsibility can be developed and tested independently.

For the final report, insert a labelled architecture diagram based on the actual implementation. Confirm the exact request and response formats by inspecting the source code before describing low-level details.

6. Technology Stack

Frontend

HTML5 provides the page structure, CSS3 controls the layout and visual appearance, and JavaScript handles interactive behaviour and communication with the backend. The README describes a responsive dark editorial interface with violet and lavender accents, animations, and glassmorphism-inspired styling.

Backend

Python is the main backend language and FastAPI is the web framework. FastAPI is used to expose endpoints and coordinate requests between the frontend, news provider, and AI provider. REST-style routes make the application's functions accessible through HTTP requests.

News provider

NewsAPI is documented as the source for retrieving recent news articles. Availability, article coverage, quotas, and response fields depend on the provider's plan and current API behaviour.

AI provider

Groq's Llama 3.3 70B model is documented as the model used for analysing and simplifying financial articles. The model receives relevant content and generates a simpler explanation.

Development tools

The README lists Git, GitHub, and Visual Studio Code among the tools used or intended for development. Dependencies are expected to be installed through requirements.txt.

7. Feature 1 — Fetch Latest Financial News

The news retrieval feature connects the application to an external news provider. Its purpose is to collect current article information so the user can begin with a relevant financial story instead of manually searching multiple sites.

How it works

1. 1. The user requests financial news through the interface.
2. 2. The frontend sends a request to the FastAPI backend.
3. 3. The backend calls NewsAPI using a server-side credential.
4. 4. The backend processes the returned article data.
5. 5. The interface displays the available news information.

Value to the user

Retrieval gives the application a current-news starting point. It also provides source information that can help users open and verify the original report. The exact sorting, filtering, article count, and fields displayed should be documented from the implementation rather than assumed.

A robust implementation should handle missing results, invalid credentials, rate limits, network timeouts, and provider errors without crashing the page.

8. Feature 2 — AI-Powered News Simplification

This is the central feature of FinNews AI. The application sends relevant article content to an AI model and requests an explanation that is easier for a general reader to understand.

Simplification goals

- Identify the main event or announcement.
- Summarise the most important information.
- Replace unnecessarily technical language with plain wording.
- Explain why a financial term or event matters, where supported by the article.
- Preserve key names, dates, amounts, and other factual details.

The README identifies Groq's Llama 3.3 70B model as the AI engine. The model's output is generated from the content and instructions supplied to it. It should not be treated as an independently verified factual record.

Why this feature matters

A beginner can receive an initial explanation without needing to understand every technical phrase in the original article. The original article should remain available so the reader can check context, qualifications, and details that may not appear in a short summary.

For best reliability, prompts and output handling should discourage invented facts and encourage the model to state when information is unclear. Human verification and testing are still importan

9. Feature 3 — Search and Category Browsing

The README lists search for financial news and browsing news by categories as application features. These functions help users locate information relevant to their interests rather than viewing only a general news feed.

Search

Search is intended to let a user enter a topic or keyword and locate related financial stories. Examples of possible topics include company announcements, banking, markets, inflation, or economic policy. The exact supported query rules depend on the implemented backend and the NewsAPI request configuration.

Categories

Category browsing groups stories by topic and can make exploration more organised. Categories should be clearly labelled and should not imply that every article has been independently classified unless that is how the source code works.

Benefits

- Makes discovery more direct.
- Helps readers focus on a topic of interest.
- Reduces the effort of scanning unrelated headlines.

When testing these features, use different keywords, empty searches, misspellings, and queries with no results. Confirm that the interface communicates the outcome clearly.

10. Feature 4 — Explain Financial Terms

The README documents a dedicated endpoint for explaining a financial term. This feature addresses a common barrier to financial literacy: readers may understand the general subject of an article but get stuck on one unfamiliar expression.

A term explanation should define the concept in plain language, avoid unnecessary jargon, and provide a short example when appropriate. For instance, a definition of inflation can explain that it is a general increase in prices over time, meaning that the same amount of money buys fewer goods and services.

The feature is useful because it allows a user to ask about a concept without needing to process a full article again. It may also support learning by helping users build familiarity with recurring financial vocabulary.

The report should describe the exact input format and output structure only after checking the endpoint implementation. The README identifies POST /explain as the endpoint but does not provide a complete schema.

Financial definitions should be educational and balanced. Where a term has multiple meanings depending on context, the response should indicate that context matters.

11. Feature 5 — Responsive Interface and User Experience

The project README describes a responsive interface, loading states, animations, and a Midnight Plum × Electric Violet visual theme. It also mentions a dark editorial design, glassmorphism, neon-pink highlights, and soft lavender elements.

Responsive layout

Responsive design aims to keep the interface usable across desktop, tablet, and mobile screens. Text should remain readable, controls should be easy to tap, and news cards or result sections should adapt to the available width.

Loading feedback

External API and AI requests may take time. A loading indicator helps users understand that a request is in progress and reduces the chance of repeated clicks caused by uncertainty.

Animations and visual hierarchy

Subtle animation can make state changes feel smoother. Strong contrast, clear headings, and consistent spacing help readers identify headlines, summaries, controls, and error messages. Animation should not interfere with readability or accessibility.

These design details are documented in the README. Before submission, verify the actual rendered interface on multiple screen sizes and add screenshots of the working application.

12. Backend API and Request Workflow

The README lists four routes: GET /, GET /health, GET /news, POST /simplify, and POST /explain. The home route is described as checking application status, /health as checking backend health, /news as retrieving current news, /simplify as generating an AI-powered summary, and /explain as explaining a financial term.

Typical workflow

6. 1. The user opens the web application.
7. 2. The frontend requests news or accepts a user query.
8. 3. FastAPI validates and processes the request.
9. 4. The news service retrieves article information when required.
10. 5. The AI service generates a simplified summary or term explanation.
11. 6. The backend returns a structured response.
12. 7. JavaScript updates the interface with results or an error message.

A clean API design should validate inputs, return meaningful status codes, avoid leaking secrets, and provide predictable response formats. API documentation can be checked through FastAPI's generated documentation interface if enabled in the running application.

Note: the README says GET / checks application status and separately lists GET /health. Confirm exact route behaviour in app/main.py before using this as a final implementation claim.

13. Security and Error Handling

The README emphasises secure API-key management and proper error handling. It documents NEWS_API_KEY and GROQ_API_KEY as environment variables and advises keeping the .env file out of version control using .gitignore.

API-key protection

- Store provider keys in environment variables on the backend.
- Do not place secret keys in browser JavaScript or public repositories.
- Keep local secret files excluded from Git.
- Rotate any key that has been exposed accidentally.

Error handling

Potential failures include missing credentials, an unavailable news provider, API quota limits, network problems, malformed responses, invalid user input, and AI-service errors. The application should display understandable messages and allow the user to retry where appropriate.

Responsible AI

A generated summary may omit important qualifications or misread a figure. Users should be encouraged to check the original source, especially when an article concerns investments, personal finances, or major economic decisions.

The README describes the project as informational and educational, not as a source of personalised financial advice or guaranteed market predictions.

14. Implementation and Setup

The README describes a Python and FastAPI backend, HTML/CSS/JavaScript frontend, NewsAPI integration, and Groq AI integration. The documented setup process is outlined below; exact dependency versions and commands should be checked against the supplied requirements file if available.

Setup outline

13.1. Extract the project into a working directory.
14.2. Install Python and create a virtual environment if appropriate.
15.3. Install project dependencies using pip install -r requirements.txt.
16.4. Create a local .env file with NEWS_API_KEY and GROQ_API_KEY.
17.5. Ensure the .env file is excluded from version control.
18.6. Start the server using uvicorn app.main:app --reload, as documented in the README.
19.7. Open the local server address in a browser and test the routes and interface.

Important limitation of the supplied ZIP: it contains only README.md, not the application source code, requirements.txt, templates, static files, or configuration files. As a result, these are documented setup instructions, not a verified installation of the uploaded archive. Include the complete project source in the final submission and validate the commands against it.

15. Testing and Validation Plan

Testing should verify that each documented feature behaves as expected and that failures are handled safely. Because the uploaded archive contains only a README, the cases below are a recommended test plan rather than evidence that tests have already passed.

Functional tests

- Open the home page and confirm that the interface loads.
- Fetch news and verify that results are displayed when the provider returns articles.
- Search with a valid keyword and check that relevant results are returned.
- Browse each available category and verify the result list.
- Submit article content to the simplification endpoint and inspect the output.
- Submit a financial term to the explanation endpoint and check clarity.

Negative and resilience tests

- Test empty input and invalid request data.
- Test missing or invalid API credentials.
- Test provider timeout, rate-limit, and no-results scenarios where feasible.
- Check that errors do not reveal API keys or sensitive configuration.

Usability tests

Test on desktop and mobile widths. Check readability, keyboard navigation, visible loading feedback, clear error messages, and the ability to find the original article. Record each test’s input, expected result, actual result, and pass/fail status in the final version.

16. Expected Results, Benefits and Limitations

Expected results

When the external services are configured and available, the application is intended to show financial news and provide simplified explanations. It is also intended to help users understand financial terms and navigate topics through search and category browsing.

Benefits

- Improves accessibility of complex financial language.
- Combines news retrieval and AI explanation in one workflow.
- Can save time when a reader needs an initial overview.
- Demonstrates practical full-stack development and AI integration.
- Encourages financial learning among beginners.

Limitations

- News coverage and freshness depend on the external provider and its plan.
- AI-generated content can be incomplete, ambiguous, or inaccurate.
- Internet access and valid API credentials are required for external services.
- The application does not replace original reporting or professional advice.
- User accounts, saved articles, history, sentiment analysis, multilingual support, and database integration are listed as future work.

Do not present unmeasured performance claims such as exact accuracy, response time, or user satisfaction unless those measurements have actually been collected.

17. Future Enhancements and Conclusion

Future enhancements

- User authentication with secure account management.
- Personalised news feeds based on selected topics.
- Saved articles and reading history.
- News sentiment analysis, with clear explanations of its limitations.
- Market trend visualisations using verified data sources.
- Multiple explanation modes, such as short summary and detailed explanation.
- Support for additional languages.
- Database integration for preferences and saved content.
- More advanced topic filtering and improved accessibility.

Conclusion

FinNews AI demonstrates how a web application can combine financial news retrieval with generative AI to make information more accessible. Its documented core features include retrieving news, simplifying articles, searching and browsing categories, and explaining financial terms. FastAPI provides the backend layer, while HTML, CSS, and JavaScript support the browser interface.

The project is valuable as an educational application and as a demonstration of REST API integration and large-language-model usage. Its reliability depends on careful handling of source data, external-service failures, API security, and AI-generated output. The original article remains essential for verification, and the application should not be presented as a personalised investment adviser.

Before final submission, add screenshots, source-code-specific implementation details, actual test results, and institution-required front matter. These additions will make the report more evidence-based and aligned with the final working project.

18. References and Appendix

Project reference

Financial-News-Simplifier-main.zip — supplied project archive. The archive available for this report contains a README.md file describing the application, features, architecture, technology stack, routes, setup, and future work.

Technology references to consult

- FastAPI official documentation: https://fastapi.tiangolo.com/
- Python official documentation: https://docs.python.org/3/
- NewsAPI documentation: https://newsapi.org/docs
- Groq developer documentation: https://console.groq.com/docs
- MDN Web Docs for HTML, CSS, and JavaScript: https://developer.mozilla.org/

Suggested appendix material

- Screenshot of the application home page.
- Screenshot showing fetched financial news.
- Screenshot of an AI-generated simplified article.
- Screenshot of the financial-term explanation feature.
- Architecture diagram and project folder structure.
- Selected source-code excerpts for each endpoint.
- Completed test-case table and known issues.

Academic integrity note: verify every feature against the actual code before submission. Clearly label proposed or future features, and do not claim test results that have not been performed.